# CDA 3103

Project Details

# Project overview

- Simulate a MIPS processor in C
  - There are three files: spimcore.c, spimcore.h, and project.c
  - Your task is to fill in the functions in project.c
  - These functions are called by the “step” function in spincore.c
  - spimcore.h contains the prototypes for the functions and the control signal structure definition

# Misconceptions

- You should not make any changes to spimcore.c or spimcore.h
- You do not require any input/output functions
  - these are handled by spimcore.c
- You do not need to convert between HEX and/or DEC

# Development Environment Options

- MUST compile with gcc
- Linux (Virtual Machine or otherwise)
  - $ gcc -o spimcore spimcore.c project.c
  - $ ./spimcore <filename>.asc

# Controls

| | |
|---|---|
| R | Show register contents |
| M | Show memory contents |
| S | Step forward 1 instruction |
| C | Continue all instructions until halt |
| H | Check to see if program has halted |
| X | Quit |

# IF

- int instruction_fetch(unsigned PC, unsigned *Mem, unsigned *instruction)

- Mem is an array and has already been populated
- PC is the current index to use for instruction memory
- Check for word alignment
- Use PC >> 2 to get the actual location (because Mem is an array of words in C, instead of an array of bytes)

# IP

- void instruction_partition(unsigned instruction, unsigned *op, unsigned *r1,unsigned *r2, unsigned *r3, unsigned *funct, unsigned *offset, unsigned *jsec)

- unsigned op, // instruction [31-26]
- r1, // instruction [25-21]
- r2, // instruction [20-16]
- r3, // instruction [15-11]
- funct, // instruction [5-0]
- offset, // instruction [15-0]
- jsec; // instruction [25-0]

- No need to check the opcode here, make copies of all sections regardless of instruction type.

# ID

- int instruction_decode(unsigned op,struct_controls *controls)

- typedef struct {
  char RegDst;
  char Jump;
  char Branch;
  char MemRead;
  char MemtoReg;
  char ALUOp;
  char MemWrite;
  char ALUSrc;
  char RegWrite;
  } struct_controls;

Based on the opcode, set the control signals in the controls structure. These are stored in the character data type, but do not have to be

characters (the data type just provides a "box" that is 8 bits wide). Most signals are either 0 or 1. ALUOp is a 3 bit signal in this project.

# Read register

- void read_register(unsigned r1,unsigned r2,unsigned *Reg,unsigned *data1,unsigned *data2)

- Reg is an array containing the register file

# Sign Extend

- void sign_extend(unsigned offset,unsigned *extended_value)

- 16th bit is the sign bit
- During instruction_partition, you may have put all zeros in the upper order 16 bits. In that case, you would only need to change the value if the sign bit is negative.

# ALU_operations

- int ALU_operations(unsigned data1,unsigned data2,unsigned extended_value,unsigned funct,char ALUOp,char ALUSrc,unsigned *ALUresult,char *Zero)

- Set parameters for A, B, and ALUControl
- If R-type instruction, look at funct
- In the end, call ALU function

# rw_memory

- int rw_memory(unsigned ALUresult,unsigned data2,char MemWrite,char MemRead,unsigned *memdata,unsigned *Mem)

- Mem is the memory array
- If MemWrite = 1, check word alignment and write into memory
- If MemRead = 1, check word alignment and read from memory

# write_register

- void write_register(unsigned r2,unsigned r3,unsigned memdata,unsigned ALUresult,char RegWrite,char RegDst,char MemtoReg,unsigned *Reg)

- If RegWrite == 1, and MemtoReg == 1, then data is coming from memory
- If RegWrite == 1, and MemtoReg == 0, then data is coming from ALU_result
- If RegWrite == 1, place write data into the register specified by RegDst

# PC_update

- void PC_update(unsigned jsec,unsigned extended_value,char Branch,char Jump,char Zero,unsigned *PC)

- PC = PC + 4;
- Take care of Branch and Jump
- Zero – Branch taken or not
- Jump: Left shift bits of jsec by 2 and use upper 4 bits of PC

# Hints

- Accessing memory
  - Mem[0] is an unsigned int, aka the whole WORD.
  - This means that given an address, like the PC, you need to shift the address to the RIGHT by 2 to form the index for the Mem structure.

# Hints

- Isolating bits in an unsigned int can be done with a bitwise AND and shift in C. For example:
  - int op = (inst & 0xFC000000) >> 26;
  - int op = (inst & 0b11111100000000000000000000000000) >> 26;
- The input is a series of HEX numbers representing instructions.
  - Don't write anything to parse this file, this is handled by spimcore.c.
  - Those numbers are already sitting in the Mem array at the proper address.
  - They do not need to be converted in order to do operations like AND and shift as shown above.

# Questions?